

TRD

Technical Requirements Document (TRD)

AWS S3-Based Data Processing Pipeline

-

TR-INGEST-001

- Related Functional Requirement(s): FR-INGEST-001
- Objective:

Implement S3 CSV ingestion for customer data using AWS Glue Spark job.

- Target Cluster Configuration:
 - AWS Glue ETL Job
 - Spark engine (Glue 3.0 or later)
 - Worker type: G.1X or G.2X
 - Number of workers: 2-10 (based on data volume)
- Source Data Details:
 - Location: `s3://adif-sdlc/sdlc_wizard/customerdata/`
 - Format: CSV
 - Schema: CustId (string), Name (string), EmailId (string), Region (string)
 - Encoding: UTF-8
 - Delimiter: comma
- Target Data Details:
 - In-memory Spark DataFrame
 - Schema preserved from source
- Job Flow / Pipeline Stages:
 1. Initialize Glue Spark session
 2. Read CSV from S3 using `spark.read.csv()`
 3. Apply schema: CustId, Name, EmailId, Region
 4. Load into DataFrame for downstream processing
- Data Transformations / Business Logic:
 - Parse CSV with header row

- Infer or enforce schema matching source columns
- Error Handling and Logging:
 - Log file path and row count to CloudWatch
 - Raise exception if file not found or schema mismatch
 - Capture parse errors for malformed rows
- Assumptions:
 - CSV file uses standard comma delimiter
 - Header row present
 - File encoding is UTF-8
-

TR-INGEST-002

- Related Functional Requirement(s): FR-INGEST-002
- Objective:

Implement S3 CSV ingestion for order data using AWS Glue Spark job.

- Target Cluster Configuration:
 - AWS Glue ETL Job
 - Spark engine (Glue 3.0 or later)
 - Worker type: G.1X or G.2X
 - Number of workers: 2-10 (based on data volume)
- Source Data Details:
 - Location: `s3://adif-sdlc/sdlc\_wizard/orderdata/`
 - Format: CSV
 - Schema: OrderId (string), ItemName (string), PricePerUnit (decimal), Qty (integer), Date (string), CustId (string)
 - Encoding: UTF-8
 - Delimiter: comma
- Target Data Details:
 - In-memory Spark DataFrame
 - Schema preserved from source
- Job Flow / Pipeline Stages:
 1. Initialize Glue Spark session

2. Read CSV from S3 using ``spark.read.csv()``
3. Apply schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
4. Load into DataFrame for downstream processing

- Data Transformations / Business Logic:
 - Parse CSV with header row
 - Infer or enforce schema matching source columns
 - Cast PricePerUnit to decimal, Qty to integer
- Error Handling and Logging:
 - Log file path and row count to CloudWatch
 - Raise exception if file not found or schema mismatch
 - Capture parse errors for malformed rows
- Assumptions:
 - CSV file uses standard comma delimiter
 - Header row present
 - File encoding is UTF-8
 - Date format is consistent (e.g., YYYY-MM-DD)
-

TR-CLEAN-001

- Related Functional Requirement(s): FR-CLEAN-001
- Objective:

Remove rows containing "Null" string or NULL values from customer DataFrame.

- Target Cluster Configuration:
 - Same Glue job as TR-INGEST-001
 - No additional cluster resources required
- Source Data Details:
 - In-memory customer DataFrame from TR-INGEST-001
- Target Data Details:
 - Cleaned customer DataFrame
 - Schema unchanged
- Job Flow / Pipeline Stages:

1. Filter DataFrame to exclude rows where any column is NULL or equals "Null"
2. Return cleaned DataFrame

- Data Transformations / Business Logic:
 - Use Spark ``filter()`` or ``dropna()`` to remove NULL values
 - Use ``filter()`` with condition to exclude string "Null" (case-insensitive)
- Error Handling and Logging:
 - Log count of rows removed to CloudWatch
 - Log final row count after cleaning
- Assumptions:
 - "Null" string matching is case-insensitive
 - Empty strings are not treated as NULL
-

TR-CLEAN-002

- Related Functional Requirement(s): FR-CLEAN-002
- Objective:

Remove rows containing "Null" string or NULL values from order DataFrame.

- Target Cluster Configuration:
 - Same Glue job as TR-INGEST-002
 - No additional cluster resources required
- Source Data Details:
 - In-memory order DataFrame from TR-INGEST-002
- Target Data Details:
 - Cleaned order DataFrame
 - Schema unchanged
- Job Flow / Pipeline Stages:
 1. Filter DataFrame to exclude rows where any column is NULL or equals "Null"
 2. Return cleaned DataFrame
- Data Transformations / Business Logic:
 - Use Spark ``filter()`` or ``dropna()`` to remove NULL values
 - Use ``filter()`` with condition to exclude string "Null" (case-insensitive)

- Error Handling and Logging:
- Log count of rows removed to CloudWatch
- Log final row count after cleaning
- Assumptions:
- "Null" string matching is case-insensitive
- Empty strings are not treated as NULL
-

TR-CLEAN-003

- Related Functional Requirement(s): FR-CLEAN-003
- Objective:

Remove duplicate rows from customer DataFrame.

- Target Cluster Configuration:
- Same Glue job as TR-CLEAN-001
- No additional cluster resources required
- Source Data Details:
- Cleaned customer DataFrame from TR-CLEAN-001
- Target Data Details:
- Deduplicated customer DataFrame
- Schema unchanged
- Job Flow / Pipeline Stages:
 1. Apply `dropDuplicates()` on all columns
 2. Return deduplicated DataFrame
- Data Transformations / Business Logic:
- Use Spark `dropDuplicates()` without column specification (applies to all columns)
- Retain first occurrence of duplicate rows
- Error Handling and Logging:
- Log count of duplicates removed to CloudWatch
- Log final row count after deduplication
- Assumptions:
- Duplicates determined by exact match across all columns

- No preference for which duplicate instance to retain
-

TR-CLEAN-004

- Related Functional Requirement(s): FR-CLEAN-004
- Objective:

Remove duplicate rows from order DataFrame.

- Target Cluster Configuration:
- Same Glue job as TR-CLEAN-002
- No additional cluster resources required
- Source Data Details:
- Cleaned order DataFrame from TR-CLEAN-002
- Target Data Details:
- Deduplicated order DataFrame
- Schema unchanged
- Job Flow / Pipeline Stages:
 1. Apply `dropDuplicates()` on all columns
 2. Return deduplicated DataFrame
- Data Transformations / Business Logic:
- Use Spark `dropDuplicates()` without column specification (applies to all columns)
- Retain first occurrence of duplicate rows
- Error Handling and Logging:
- Log count of duplicates removed to CloudWatch
- Log final row count after deduplication
- Assumptions:
- Duplicates determined by exact match across all columns
- No preference for which duplicate instance to retain
-

TR-CATALOG-001

- Related Functional Requirement(s): FR-CATALOG-001

- Objective:

Register cleaned customer data as Glue Catalog table using Glue API or Spark.

- Target Cluster Configuration:
 - Same Glue job as TR-CLEAN-003
 - Glue Catalog API access enabled
- Source Data Details:
 - Deduplicated customer DataFrame from TR-CLEAN-003
- Target Data Details:
 - Database: ``gen_ai_poc_databrickscoe``
 - Table: ``sdlc_wizard_customer``
 - Schema: CustId (string), Name (string), EmailId (string), Region (string)
 - Storage format: Parquet (recommended) or CSV
 - Location: Managed by Glue or explicit S3 path
- Job Flow / Pipeline Stages:
 1. Write DataFrame to S3 in Parquet format
 2. Use ``glueContext.create_dynamic_frame.from_catalog()`` or Glue API to register table
 3. Update Glue Catalog metadata
- Data Transformations / Business Logic:
 - Convert DataFrame to DynamicFrame if using GlueContext
 - Define table schema matching source columns
- Error Handling and Logging:
 - Log table registration success/failure to CloudWatch
 - Raise exception if database does not exist
 - Handle table already exists scenario (update or skip)
- Assumptions:
 - Glue database ``gen_ai_poc_databrickscoe`` exists
 - IAM role has ``glue:CreateTable`` and ``glue:UpdateTable`` permissions
 - Table location managed by Glue unless explicit path provided
-

TR-CATALOG-002

- Related Functional Requirement(s): FR-CATALOG-002

- Objective:

Register cleaned order data as Glue Catalog table using Glue API or Spark.

- Target Cluster Configuration:
 - Same Glue job as TR-CLEAN-004
 - Glue Catalog API access enabled
- Source Data Details:
 - Deduplicated order DataFrame from TR-CLEAN-004
- Target Data Details:
 - Database: ``gen_ai_poc_databrickscoe``
 - Table: ``sdlc_wizard_order``
 - Schema: OrderId (string), ItemName (string), PricePerUnit (decimal), Qty (integer), Date (string), CustId (string)
 - Storage format: Parquet (recommended) or CSV
 - Location: Managed by Glue or explicit S3 path
- Job Flow / Pipeline Stages:
 1. Write DataFrame to S3 in Parquet format
 2. Use ``glueContext.create_dynamic_frame.from_catalog()`` or Glue API to register table
 3. Update Glue Catalog metadata
- Data Transformations / Business Logic:
 - Convert DataFrame to DynamicFrame if using GlueContext
 - Define table schema matching source columns
- Error Handling and Logging:
 - Log table registration success/failure to CloudWatch
 - Raise exception if database does not exist
 - Handle table already exists scenario (update or skip)
- Assumptions:
 - Glue database ``gen_ai_poc_databrickscoe`` exists
 - IAM role has ``glue:CreateTable`` and ``glue:UpdateTable`` permissions
 - Table location managed by Glue unless explicit path provided
-

TR-SCD-001

- Related Functional Requirement(s): FR-SCD-001
- Objective:

Implement SCD Type 2 logic for ordersummary table using Apache Hudi on S3.

- Target Cluster Configuration:
 - AWS Glue ETL Job
 - Spark engine with Hudi libraries (Glue 3.0 or later)
 - Worker type: G.1X or G.2X
 - Number of workers: 5-20 (based on data volume)
 - Hudi connector JAR included in job dependencies
- Source Data Details:
 - Customer table: ``gen_ai_poc_databrickscoe.sdlic_wizard_customer``
 - Order table: ``gen_ai_poc_databrickscoe.sdlic_wizard_order``
- Target Data Details:
 - Location: ``s3://adif-sdlic/curated/sdlic_wizard/ordersummary/``
 - Format: Apache Hudi (COW or MOR)
 - Schema: CustId, Name, EmailId, Region, OrderId, ItemName, PricePerUnit, Qty, Date, IsActive (boolean), StartDate (timestamp), EndDate (timestamp), OpTs (timestamp)
 - Primary key: OrderId
 - Precombine key: OpTs
- Job Flow / Pipeline Stages:
 1. Read customer and order tables from Glue Catalog
 2. Join DataFrames on CustId
 3. Detect changes by comparing with existing Hudi table (if exists)
 4. For new/changed records:
 - Set IsActive = true, StartDate = current timestamp, EndDate = null, OpTs = current timestamp
 5. For existing records being updated:
 - Update old record: IsActive = false, EndDate = current timestamp, OpTs = current timestamp
 6. Write to S3 using Hudi upsert operation
 7. Sync Hudi metadata to Glue Catalog
- Data Transformations / Business Logic:
 - Join: ``customer.join(order, customer.CustId == order.CustId, "inner")``

- Add SCD columns: IsActive, StartDate, EndDate, OpTs
- Change detection: compare joined result with existing Hudi table on non-metadata columns
- Hudi write options:
 - ``hoodie.datasource.write.operation`: `upsert``
 - ``hoodie.datasource.write.recordkey.field`: `OrderId``
 - ``hoodie.datasource.write.precombine.field`: `OpTs``
 - ``hoodie.datasource.write.table.name`: `ordersummary``
 - ``hoodie.table.name`: `ordersummary``
- Error Handling and Logging:
 - Log join row count to CloudWatch
 - Log Hudi commit metadata (commit time, records written)
 - Raise exception on Hudi write failure
 - Validate schema before write
- Assumptions:
 - Change detection based on all non-metadata columns (CustId, Name, EmailId, Region, OrderId, ItemName, PricePerUnit, Qty, Date)
 - OpTs generated using ``current_timestamp()``
 - Hudi table type: Copy-On-Write (COW) unless specified
 - No backdated changes; all updates are forward-dated
-

TR-SCD-002

- Related Functional Requirement(s): FR-SCD-002
- Objective:

Register Hudi ordersummary table in Glue Catalog.

- Target Cluster Configuration:
 - Same Glue job as TR-SCD-001
 - Glue Catalog API access enabled
- Source Data Details:
 - Hudi table at ``s3://adif-sdlc/curated/sdlc_wizard/ordersummary/``
- Target Data Details:
 - Database: ``gen_ai_poc_databrickscoe``

- Table: `ordersummary`
- Schema: CustId, Name, EmailId, Region, OrderId, ItemName, PricePerUnit, Qty, Date, IsActive, StartDate, EndDate, OpTs
- Storage format: Hudi
- Location: `s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/`
- Job Flow / Pipeline Stages:
 1. Use Hudi sync tool or Glue API to register table
 2. Set table properties for Hudi format
 3. Update Glue Catalog metadata
- Data Transformations / Business Logic:
 - Use Hudi `HoodieDeltaStreamer` or Glue Catalog API
 - Set SerDe properties for Hudi input format
- Error Handling and Logging:
 - Log table registration success/failure to CloudWatch
 - Raise exception if database does not exist
 - Handle table already exists scenario (update metadata)
- Assumptions:
 - Glue Catalog supports Hudi table format (Glue 2.0+)
 - IAM role has `glue:CreateTable` and `glue:UpdateTable` permissions
 - Athena can query Hudi tables via Glue Catalog
-

TR-INCR-001

- Related Functional Requirement(s): FR-INCR-001
- Objective:

Implement incremental SCD Type 2 updates triggered by customer data changes.

- Target Cluster Configuration:
 - Same Glue job as TR-SCD-001
 - Triggered by S3 event or scheduled Glue trigger
- Source Data Details:
 - Updated customer records (delta or full snapshot)
 - Existing ordersummary Hudi table

- Target Data Details:
- Updated ordersummary Hudi table at `s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/`
- Job Flow / Pipeline Stages:
 1. Identify changed or new customer records (compare with previous snapshot or use change data capture)
 2. Filter order records associated with changed customers (join on CustId)
 3. Apply SCD Type 2 logic (same as TR-SCD-001) only for affected records
 4. Upsert to Hudi table
- Data Transformations / Business Logic:
 - Change detection: compare current customer data with previous snapshot using hash or timestamp
 - Filter orders: `order.filter(order.CustId.isin(changed\_customer\_ids))`
 - Apply SCD Type 2 logic for filtered records
 - Hudi upsert operation updates only affected records
- Error Handling and Logging:
 - Log count of changed customers and affected orders to CloudWatch
 - Log Hudi commit metadata
 - Raise exception on Hudi write failure
- Assumptions:
 - Change detection mechanism: timestamp-based or hash comparison
 - Previous customer snapshot available for comparison
 - Only affected records processed to optimize performance
-

TR-AGG-001

- Related Functional Requirement(s): FR-AGG-001
- Objective:

Generate customer aggregate spend table by summarizing total spending per customer.

- Target Cluster Configuration:
 - AWS Glue ETL Job
 - Spark engine (Glue 3.0 or later)
 - Worker type: G.1X or G.2X
 - Number of workers: 2-10 (based on data volume)

- Source Data Details:
 - Table: `gen\_ai\_poc\_databricksco.ordersummary`
 - Filter: IsActive = true
- Target Data Details:
 - Location: `s3://adif-sdlc/analytics/customeraggregatespend/`
 - Format: Parquet
 - Schema: Name (string), TotalAmount (decimal), Date (string)
- Job Flow / Pipeline Stages:
 1. Read ordersummary table from Glue Catalog
 2. Filter for active records (IsActive = true)
 3. Calculate TotalAmount = PricePerUnit Qty
 4. Group by Name and Date
 5. Aggregate: sum(TotalAmount)
 6. Write to S3 in Parquet format
- Data Transformations / Business Logic:
 - Filter: `ordersummary.filter(ordersummary.IsActive == true)`
 - Calculate: `withColumn("TotalAmount", col("PricePerUnit") col("Qty"))`
 - Group: `groupBy("Name", "Date")`
 - Aggregate: `agg(sum("TotalAmount").alias("TotalAmount"))`
- Error Handling and Logging:
 - Log row count before and after aggregation to CloudWatch
 - Raise exception on write failure
 - Validate TotalAmount is non-negative
- Assumptions:
 - TotalAmount calculation: PricePerUnit Qty (no taxes or discounts)
 - Date represents transaction date from order data
 - Active records only (IsActive = true)
-

TR-AGG-002

- Related Functional Requirement(s): FR-AGG-002
- Objective:

Register customeraggregatespend table in Glue Catalog.

- Target Cluster Configuration:
- Same Glue job as TR-AGG-001
- Glue Catalog API access enabled

- Source Data Details:
- Parquet files at `s3://adif-sdlc/analytics/customeraggregatespend/`

- Target Data Details:
- Database: `gen\_ai\_poc\_databrickscoe`
- Table: `customeraggregatespend`
- Schema: Name (string), TotalAmount (decimal), Date (string)
- Storage format: Parquet
- Location: `s3://adif-sdlc/analytics/customeraggregatespend/`

- Job Flow / Pipeline Stages:
 1. Write DataFrame to S3 in Parquet format
 2. Use Glue API or GlueContext to register table
 3. Update Glue Catalog metadata

- Data Transformations / Business Logic:
- Convert DataFrame to DynamicFrame if using GlueContext
- Define table schema matching aggregated columns

- Error Handling and Logging:
- Log table registration success/failure to CloudWatch
- Raise exception if database does not exist
- Handle table already exists scenario (update or skip)

- Assumptions:
- Glue database `gen\_ai\_poc\_databrickscoe` exists
- IAM role has `glue:CreateTable` and `glue:UpdateTable` permissions
- Parquet format compatible with Athena queries
-

TR-ARCH-001

- Related Functional Requirement(s): FR-ARCH-001

- Objective:

Execute all ETL operations using AWS Glue with Spark and Hudi.

- Target Cluster Configuration:
 - AWS Glue ETL Jobs (Python Shell or Spark)
 - Glue version: 3.0 or later
 - Spark engine with Hudi connector
 - Worker types: G.1X, G.2X, or G.4X based on workload
 - Auto-scaling enabled (min 2, max 10 workers)
- Source Data Details:
 - S3 input paths for customer and order data
 - Glue Catalog tables for intermediate and curated data
- Target Data Details:
 - S3 output paths for curated and analytics data
 - Glue Catalog tables for all processed datasets
- Job Flow / Pipeline Stages:
 1. Define Glue jobs for ingestion, cleaning, transformation, SCD, aggregation
 2. Configure job dependencies and triggers
 3. Execute jobs using Spark engine
 4. Use Hudi for SCD Type 2 operations
- Data Transformations / Business Logic:
 - Modularize ETL stages into separate Glue jobs or job steps
 - Use Glue workflows to orchestrate job execution
 - Leverage Spark DataFrames and Hudi APIs
- Error Handling and Logging:
 - Enable CloudWatch logging for all Glue jobs
 - Set job bookmarks for incremental processing
 - Configure retry logic for transient failures
- Assumptions:
 - Glue job configurations include Hudi JAR dependencies
 - Sufficient Glue DPU capacity allocated
 - IAM roles grant necessary S3, Glue Catalog, and CloudWatch permissions
-

TR-ARCH-002

- Related Functional Requirement(s): FR-ARCH-002

- Objective:

Use AWS S3 as storage layer for all data.

- Target Cluster Configuration:
- N/A (S3 is serverless)
- Source Data Details:
- Input paths: `s3://adif-sdlc/sdlc\_wizard/customerdata/`, `s3://adif-sdlc/sdlc\_wizard/orderdata/`
- Target Data Details:
- Curated path: `s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/`
- Analytics path: `s3://adif-sdlc/analytics/customeraggregatespend/`
- Formats: CSV (input), Parquet (intermediate/analytics), Hudi (curated)
- Job Flow / Pipeline Stages:
 1. Read source data from S3 input paths
 2. Write processed data to S3 curated and analytics paths
 3. Maintain data in appropriate formats
- Data Transformations / Business Logic:
- Use Spark `read` and `write` APIs for S3 I/O
- Configure S3 paths in Glue job parameters
- Error Handling and Logging:
- Log S3 read/write operations to CloudWatch
- Raise exception on S3 access errors (permissions, path not found)
- Validate S3 paths before job execution
- Assumptions:
- S3 bucket versioning and encryption configured per security policies
- IAM roles grant `s3:GetObject`, `s3:PutObject`, `s3:ListBucket` permissions
- Data retention policies defined for lifecycle management
-

TR-ARCH-003

- Related Functional Requirement(s): FR-ARCH-003

- Objective:

Enable SQL-based querying via Amazon Athena.

- Target Cluster Configuration:

- Amazon Athena (serverless)
- Athena workgroup configured with query result location

- Source Data Details:

- Glue Catalog tables: `sdlc\_wizard\_customer`, `sdlc\_wizard\_order`, `ordersummary`, `customeraggregatespend`

- Target Data Details:

- Query results stored in S3 (Athena workgroup result location)

- Job Flow / Pipeline Stages:

1. Users execute SQL queries via Athena console, API, or JDBC/ODBC
2. Athena reads table metadata from Glue Catalog
3. Athena scans data from S3 locations
4. Query results returned to user or written to S3

- Data Transformations / Business Logic:

- Standard SQL queries supported (SELECT, JOIN, WHERE, GROUP BY, etc.)
- Athena supports Parquet, Hudi, CSV formats

- Error Handling and Logging:

- Athena query logs available in CloudWatch
- Query failures logged with error messages
- Monitor query performance and costs via CloudWatch metrics

- Assumptions:

- Athena workgroup configured with S3 result location
- IAM roles grant `athena:StartQueryExecution`, `athena:GetQueryResults` permissions
- Glue Catalog tables registered and accessible

•

TR-ARCH-004

- Related Functional Requirement(s): FR-ARCH-004

- Objective:

Monitor ETL jobs using Amazon CloudWatch.

- Target Cluster Configuration:
 - Amazon CloudWatch (serverless)
 - CloudWatch Logs for Glue job logs
 - CloudWatch Metrics for job performance
- Source Data Details:
 - Glue job execution logs and metrics
- Target Data Details:
 - CloudWatch Logs: `/aws-glue/jobs/logs-v2/`
 - CloudWatch Metrics: Glue job duration, DPU usage, success/failure counts
- Job Flow / Pipeline Stages:
 1. Enable continuous logging for Glue jobs
 2. Glue jobs emit logs to CloudWatch Logs
 3. Metrics tracked: job duration, DPU hours, success/failure status
 4. Configure CloudWatch alarms for job failures or performance thresholds
- Data Transformations / Business Logic:
 - Use CloudWatch Logs Insights for log analysis
 - Create CloudWatch dashboards for job monitoring
 - Set alarms for job failures, long-running jobs, or high DPU usage
- Error Handling and Logging:
 - All Glue job errors logged to CloudWatch
 - Alarms trigger SNS notifications on failures
 - Retain logs per defined retention policy (e.g., 30 days)
- Assumptions:
 - CloudWatch logging enabled for all Glue jobs
 - IAM roles grant `logs:CreateLogGroup`, `logs:CreateLogStream`, `logs:PutLogEvents` permissions
 - CloudWatch alarms configured by operations team
-

TR-ARCH-005

- Related Functional Requirement(s): FR-ARCH-005

- Objective:

Optionally implement data governance using AWS Lake Formation.

- Target Cluster Configuration:
 - AWS Lake Formation (serverless)
 - Lake Formation permissions integrated with Glue Catalog
- Source Data Details:
 - Glue Catalog databases and tables
- Target Data Details:
 - Lake Formation permissions and policies
- Job Flow / Pipeline Stages:
 1. Register Glue Catalog with Lake Formation
 2. Define Lake Formation permissions for databases and tables
 3. Enforce fine-grained access control (column-level, row-level)
 4. Audit data access via Lake Formation logs
- Data Transformations / Business Logic:
 - Lake Formation permissions override IAM policies for Glue Catalog access
 - Define grants for users, roles, and groups
 - Use Lake Formation blueprints for automated data ingestion (optional)
- Error Handling and Logging:
 - Lake Formation access logs available in CloudWatch or S3
 - Access denied errors logged for unauthorized queries
 - Monitor permission changes via CloudTrail
- Assumptions:
 - Lake Formation enabled in AWS account
 - Data governance policies defined by data governance team
 - Integration with Glue Catalog configured
 - IAM roles grant `lakeformation:GrantPermissions`, `lakeformation:RevokePermissions` permissions
-

Global Technical Assumptions

- All AWS services deployed in same region (e.g., us-east-1) for optimal performance

- IAM roles follow least-privilege principle with explicit permissions for S3, Glue, Athena, CloudWatch, Lake Formation
- Data formats: CSV (input), Parquet (intermediate/analytics), Hudi (curated SCD Type 2)
- Network: VPC endpoints configured for S3 and Glue if using private subnets
- Batch processing model; no real-time streaming requirements
- Date fields use ISO 8601 format (YYYY-MM-DD) or consistent format across datasets
- CustId is primary key for customer data and foreign key in order data
- OrderId is primary key for order data
- TotalAmount calculation: PricePerUnit Qty (no taxes or discounts)
- SCD Type 2 logic: no backdated changes; all updates forward-dated with current timestamp
- Hudi table type: Copy-On-Write (COW) unless specified
- Glue job bookmarks enabled for incremental processing where applicable
- S3 bucket encryption: SSE-S3 or SSE-KMS per security policy
- CloudWatch log retention: 30 days (configurable)
- Athena query result location: `s3://adif-sdlc/athena-results/`